

RESOURCE CENTER ([HTTP://MAMMOTHDATA.COM/RESOURCE-CENTER/](http://MAMMOTHDATA.COM/RESOURCE-CENTER/))

PARTNERS ([HTTP://MAMMOTHDATA.COM/PARTNERS/](http://MAMMOTHDATA.COM/PARTNERS/))

CAREERS ([HTTP://MAMMOTHDATA.COM/CAREERS/](http://MAMMOTHDATA.COM/CAREERS/))

NEWS ([HTTP://MAMMOTHDATA.COM/BLOG-NEWS/](http://MAMMOTHDATA.COM/BLOG-NEWS/))

OUR TEAM ([HTTP://MAMMOTHDATA.COM/TEAM/](http://MAMMOTHDATA.COM/TEAM/))

BLOG ([HTTP://MAMMOTHDATA.COM/BLOG](http://MAMMOTHDATA.COM/BLOG))



([https://www.linkedin.com/company/open-software-](https://www.linkedin.com/company/open-software-integrators-)



integrators-

(<http://www.facebook.com/open-software-integrators->



**MAMMOTH
DATA**

([HTTP://MAMMOTHDATA.COM/](http://MAMMOTHDATA.COM/))



BLOG

DAG vs MapReduce

The new generation of Big Data tools largely focus on improving support for real-time (or near-time) computation and interactive applications by reducing the latency involved in processing jobs.

If you look at Storm, Spark, Tez, and other newer tools, you will frequently encounter the term “DAG” or Directed Acyclic Graph. This article will explain why traditional MapReduce is subject to undesirable latencies, what a DAG is, and why these new systems use this approach.

Hadoop, which began life specifically as an implementation of the MapReduce paradigm, has traditionally relied on MapReduce as its primary programming model. Hadoop MapReduce jobs display high latencies as a result of the programming model of traditional MapReduce, in which jobs follow a stock structure of “map,” followed by “shuffle,” followed by “reduce” steps. Even single-step jobs under MapReduce tend to feature high latencies. This problem is exacerbated for more complex processing involving “chaining” successive MapReduce jobs. In multi-step jobs, each job is blocked from beginning until all of the preceding jobs have finished. As a result of this model, complex computations can require time on the order of minutes, hours, or longer – even with fairly small data volumes.

A Directed Acyclic Graph, in this context, refers to a model for scheduling work in which jobs are represented as vertices in a graph, where the order of execution is specified by the directionality of the edges in the graph. The “acyclic” part just means that there are no loops (“cycles”) in the graph. In a system which schedules jobs using a DAG, independent nodes (computational steps) in the graph can run in parallel, rather than sequentially. This approach makes it easier for programmers to build more complex multi-step computations, and avoids the scheduling overhead imposed by traditional MapReduce.

Of course simply switching to a DAG for scheduling does not alleviate the high latencies associated with single-step Hadoop MapReduce jobs. This is why even workflows constructed as DAGs that link Hadoop MapReduce jobs, still suffer in the latency area. An example of this problem would be using external scheduler like Oozie to control a series of MapReduce jobs. Each workflow still has to pay the cost of high startup times and high latencies for individual jobs. So in order to achieve low overall latency, systems such as Spark, Storm, Samza, and others have also added other optimizations – primarily copying data into memory and performing substantially less disk I/O.

Aside from improving latency, DAG based systems have other advantages. For example, it is simpler to implement a fault tolerant approach using a DAG. In the event of a job failure, you can easily backtrack through the graph and re-execute any failed jobs, even at intermediate stages of a computation. The enforced order of the graph always allows you to walk through the graph from any node, to the eventual end.

Finally, we would be remiss in not pointing out that Hadoop has also moved beyond its historical reliance on simple MapReduce as well. The Hadoop 2.x series has refactored the resource allocation and scheduling components to support a much more flexible architecture, which allows the implementation of new, non MapReduce, programming models. With Hadoop 2 other processing engines can layer on top of YARN and provide low-latency, real-time processing, while living side-by-side with jobs written for MapReduce, MPI, BSP, or other execution models. Spark, in fact, can be deployed onto an existing Hadoop cluster, and take advantage of YARN for scheduling and resource allocation.



As you can see, a Directed Acyclic Graph approach is a key element of most next-generation, real-time Big Data platforms. These tools, including Storm, Spark, Samza and Tez, offer amazing new capabilities for building highly interactive, real-time computing systems to power your real-time BI, predictive analytics, real-time marketing and other critical systems.

Are you looking to incorporate a new generation of Big Data tools to support real-time computation and interactive applications? Interested in Hadoop or expanding into the Hadoop ecosystem to give your organization the data-driven success stories it needs. Give us a call at 919.321.0119 or shoot us an email info@mammothdata.com to get started.

– Phil Rhodes, Senior Consultant

Leave a Reply

Your email address will not be published.

Name



Email



Website



Comment

Submit



(<https://clutch.co/research/big-data-consultants>)



Copyright © 2015 • Mammoth Data, Inc. • All rights reserved

Contact (<http://mammothdata.com/contact/>)
| Privacy Policy (<http://mammothdata.com/privacy/>)